

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 7.2b: The Full Transformer Layer Architecture

Zhigang Feng

Spring 2026

Topic 7.2b Agenda

1. **Full Transformer Layer** — residual + LayerNorm + FFN
2. **Encoder, Decoder, Encoder–Decoder** — BERT, GPT, T5; the Vaswani 2017 diagram
3. **Appendix — research case study** — Kim, Muhn, Nikolaev, Zhang (2024): paragraph-level attention over 10-K filings

Arc: T1a Why+Tokens → T1b Embeddings → T2a Attention → T2b Transformer → T3a Training → T3b Limits → T4 Applications.

Full Transformer Layer

Residual Connection and Layer Normalization

- After the attention output $\tilde{e}^{(i)}$, the layer combines it with the original input via a *residual connection* and then a *layer normalization*:

$$u^{(i)} = \text{Norm}(e^{(i)} + \tilde{e}^{(i)}).$$

- **Why residual?**

- Preserves the original information — attention can only *add* to it, never destroy it
- Provides a clean gradient path back through dozens of layers (the trick that made deep networks trainable)

- **Why layer normalization?**

- Normalizes activations across the feature dimension to mean 0, variance 1
- Adds learnable scale γ and shift β
- Stabilizes optimization dynamics, especially for very deep stacks

Feed-Forward Network (MLP)

- **Division of labor inside a layer.** Attention mixes information *across* tokens; the FFN then transforms *each* token individually given what it just gathered. Attention = the meeting room; FFN = desk work after the meeting.
- **Why it exists.** Attention alone is a linear map of value vectors. The FFN supplies the *non-linear* transformation that lets the network represent arbitrary functions of a token's contextualized embedding. Empirically, FFN weights store most of the model's factual knowledge — delete them and attention-only models lose world facts even if the attention pattern is intact.
- After attention + residual + norm, each position is passed independently through a small feed-forward network:

$$f^{(i)} = W_2 \sigma(W_1 u^{(i)} + b_1) + b_2.$$

- **Shape conventions:** $W_1: d \rightarrow 4d$ (expand), activation σ (ReLU / GELU / SwiGLU), $W_2: 4d \rightarrow d$ (contract). The $4d$ expansion is where most of an LLM's parameters live — capacity for richer non-linear transformations.
- Followed by another residual connection and layer normalization:

$$e'^{(i)} = \text{Norm}(u^{(i)} + f^{(i)}).$$

One Layer — Putting It All Together

- A single Transformer layer can be summarized by the flow:

$$e^{(i)} \xrightarrow{\text{Self-Attention}} \tilde{e}^{(i)} \xrightarrow{\text{Res+Norm}} u^{(i)} \xrightarrow{\text{MLP}} f^{(i)} \xrightarrow{\text{Res+Norm}} e'^{(i)}.$$

- Operationally:

$$\tilde{e}^{(i)} = W_O \sum_j \alpha_j^{(i)} V_j,$$

$$u^{(i)} = \text{Norm}(e^{(i)} + \tilde{e}^{(i)}),$$

$$f^{(i)} = W_2 \sigma(W_1 u^{(i)} + b_1) + b_2,$$

$$e'^{(i)} = \text{Norm}(u^{(i)} + f^{(i)}).$$

- Same $(\mathbb{R}^d)^n$ shape in and out: a layer is a structured non-linear update.

After the Transformer: Where We Are

- **What went in:** a sequence of n tokens, converted to embeddings + positional codes, $[e^{(1)}, \dots, e^{(n)}]$ with each $e^{(i)} \in \mathbb{R}^d$.
- **What comes out of the top layer:** n *contextualized* vectors $[e'^{(1)}, \dots, e'^{(n)}]$, same shape, same dimension. Each $e'^{(i)}$ encodes, in its d coordinates,
 - (a) the identity of token $t^{(i)}$,
 - (b) its syntactic / semantic role in *this* sequence,
 - (c) what every other token contributed to its meaning (causally masked in GPT-style models).
- **What we have NOT yet done:** no token generated, no class assigned, no number returned. Contextualized embeddings are an **intermediate data structure**, not the final answer.
- **What comes next:** a small *task head* on top — a vocabulary projection (generation) or a class projection (classification) — turns the embeddings into the output we want. The next slide is that final step.

Stacking L Layers

- The output of one layer becomes the input of the next:

$$[e^{(1)}, \dots, e^{(n)}]_{\text{layer } \ell+1} = [e'^{(1)}, \dots, e'^{(n)}]_{\text{layer } \ell}.$$

- After L layers, each token's vector has accumulated context from the entire sequence (subject to causal masking, in the autoregressive case).
- **Different layers learn different things** (empirically):
 - Lower layers: surface syntax, local agreement, morphology
 - Middle layers: phrase- and clause-level structure
 - Upper layers: discourse, factual associations, task-relevant abstractions
- **What we do with the top layer (“task heads”).**
 - **Generation:** logits = $W_{\text{vocab}} e'^{(n)}$, then $P(\text{next} = w) = \text{softmax}(\text{logits})_w$. Training loss:
 $\mathcal{L} = -\sum_i \log P(t^{(i+1)} \mid t^{(1:i)})$ — the same cross-entropy that shaped every weight in the stack during pretraining.
 - **Classification:** logits = $W_{\text{class}} e'^{([\text{CLS}])}$, then $P(\text{class} = c) = \text{softmax}(\text{logits})_c$. Training loss:
 $\mathcal{L} = -\sum \log P(y_{\text{true}} \mid x)$ on a labeled set (fine-tuned encoders: BERT, CentralBankRoBERTa, ...).

Transformer vs. Word2Vec

Property	Word2Vec	Transformer
Representation	1 static vector / word	contextual vector / token
Architecture	2-layer Skip-gram / CBOW	multi-head attention + MLP, deep stack
Context scope	fixed local window (± 5)	full sequence, every token sees every token
Position info	none	explicit positional encodings
After training	lookup table; model discarded	model <i>is</i> the network used downstream
Typical size	$\sim 10^7$ params (CPU OK)	10^8 – 10^{12} params (GPU required)
Best for	similarity, feature inputs	generation, classification, QA, RAG

The big shift: from a frozen lookup table of meanings to a deep, dynamic model where meaning emerges from interaction with context.

Encoder, Decoder, Encoder–Decoder

Three Variants of the Same Block

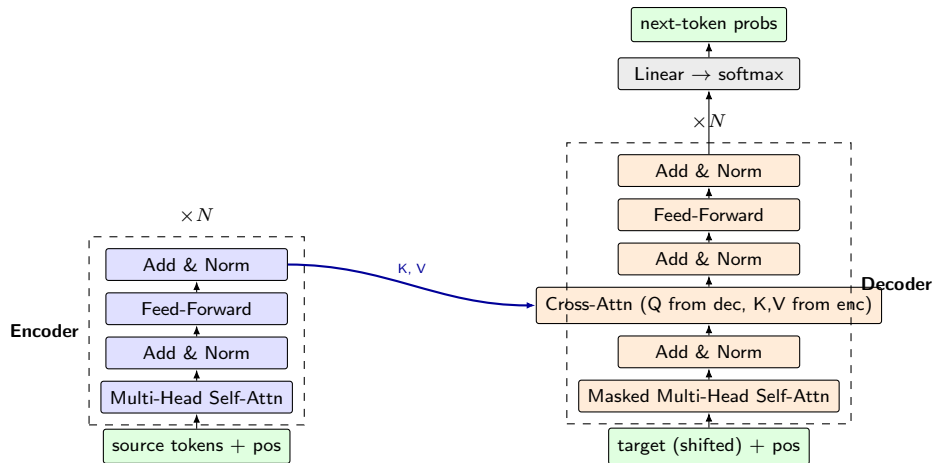
- The Transformer block is generic. The *masking pattern* and the *stack arrangement* determine what kind of model you get.
- **Encoder-only (BERT, RoBERTa, FinBERT, CentralBankRoBERTa)**
 - Bidirectional attention — every token sees every other.
 - Pretrained with *masked language modeling*: hide tokens, predict from both sides.
 - Best for: classification, embedding extraction, similarity. *Not* a generator.
- **Decoder-only (GPT, LLaMA, Claude, Qwen)**
 - Causal mask — each position only attends to itself and the past.
 - Pretrained with *next-token prediction*; autoregressive at inference (Topic 7.3a).
 - Best for: open-ended generation, in-context learning, chat. The dominant paradigm today.
- **Encoder–Decoder (T5, BART, original Vaswani 2017)**
 - Encoder reads source bidirectionally; decoder generates target with causal + cross-attention to encoder.
 - Best for: translation, summarization — anything where input and output are clearly separated sequences.

Picking a Variant for Your Macro Project

Task	Pick	Why	Example model
Sentiment / hawk–dove score on FOMC text	encoder	need a fixed embedding	CentralBankRoBERTa
Topic / event classification	encoder	supervised fine-tune	FinBERT, RoBERTa
Document embeddings for similarity / RAG	encoder	pooled $e'([CLS])$	SBERT, BGE, E5
Summarize a 10-K / press release	enc–dec	input $\neq$ output	T5, BART
Generate a forecast narrative	decoder	open-ended text	GPT-4, Claude, LLaMA
Q&A over central-bank corpora (RAG)	decoder + retriever	generative w/ context	Lecture 8

- **Rule of thumb.** If the output is a label or a vector, use an encoder. If the output is text, use a decoder. If you need to map one sequence to another and they have different lengths and structure, use encoder–decoder.

“Attention Is All You Need” (Vaswani et al., 2017) — the Architecture

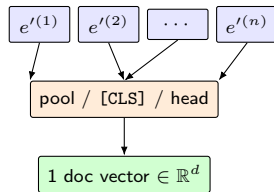


Same block, two arrangements. Decoder-only models drop the encoder and the cross-attention.

Token Embeddings vs. Document Embeddings (Bridge to Lec08)

- **What the Transformer gives us directly.** For an n -token passage, the top layer outputs n contextualized vectors $e'(1), \dots, e'(n) \in \mathbb{R}^d$ — *one per token*.
- **To compare whole passages we need one vector per passage.** Standard pooling moves:
 - **Mean-pool:** $\bar{e} = \frac{1}{n} \sum_i e'^{(i)}$ — simple, surprisingly robust.
 - **[CLS]-pool** (BERT family): use $e'^{([\text{CLS}])}$, trained to summarize the passage.
 - **Learned pooling head:** a small attention head over the token sequence, trained for the downstream objective.
- **Retrieval embedders** (Sentence-BERT, BGE, E5, voyage-3) take a pretrained transformer and fine-tune it with a *contrastive loss* on (anchor, positive, negative) triples: pull related passages together, push unrelated ones apart. The geometry is *learned for similarity*, not inherited from the

n token vectors



Lec08: ranks $\cos(\text{query}, \text{doc})$

Application: Learning Fundamentals from 10-K Text

From Tokens to Paragraphs: A Research Question

- **Question.** Which language inside a 10-K filing actually moves stock prices? Press releases, MD&A, risk factors, governance disclosures — which content do investors *react* to?
- **Paper.** Kim, Muhn, Nikolaev, Zhang (Booth WP, Nov. 2024), *Learning Fundamentals from Text*.
- **The move.** Same Q/K/V machinery from Topic 7.2a — but the unit of analysis is a **paragraph**, not a word token. A 10-K is a sequence of ~ 400 paragraphs; treat each one as a “token” and let attention learn which paragraphs matter for the market.
- **Why this beats the standard playbook:**
 - Prior text-in-finance work collapses a 10-K to one number: sentiment (Loughran–McDonald 2011), uncertainty (Hassan et al. 2019). Information loss is severe.
 - Attention preserves *paragraph-level* granularity *and* learns relational importance directly from market reactions.
 - Trained attention weights are inspectable — you can read off which content the market focuses on.

Architecture: Two Attention Layers + MLP Head

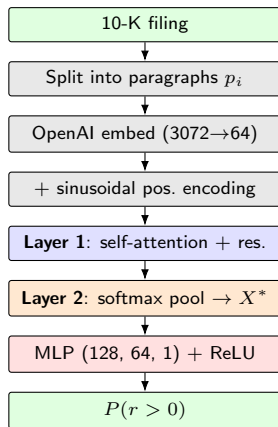
Pipeline (full universe of EDGAR 10-Ks, 1996–2023, ~77K filings, ~20.7M paragraphs):

1. Split filing $\rightarrow$ paragraphs $p_1, \dots, p_n$ (cap $n=1000$).
2. $e_i = \text{OpenAI text-embedding-3-large}(p_i)$, then truncate to first 64 dims (Matryoshka, Kusupati et al. 2022).
3. Stack $X = [e_1; \dots; e_n] \in \mathbb{R}^{n \times 64}$, add sinusoidal positional encoding.
4. **Layer 1** — Vaswani self-attention with residual: $\text{Attention}(X) \in \mathbb{R}^{n \times 64}$.
5. **Layer 2** — learned softmax pool to a doc vector:

$$X^* = \text{softmax}\left(\frac{W^\top \text{Attention}(X)^\top}{\sqrt{64}}\right) \text{Attention}(X)$$

with $W \in \mathbb{R}^{64}$ learnable; $X^* \in \mathbb{R}^{64}$.

6. MLP ($64 \rightarrow 128 \rightarrow 64 \rightarrow 1$), ReLU, sigmoid
 $\rightarrow P(r_{[-1, +30]} > 0)$.



Target: $\mathbb{I}\{r_{[\text{file}-1, \text{file}+30]} > 0\}$. Rolling 6/2/1-yr train/val/test.

Every Concept from This Topic, Reused

Slide concept (this topic)	Paper usage
Q, K, V projections (7.2a)	Same, but over <i>paragraphs</i> not tokens
Scaled softmax $\text{softmax}(QK^\top / \sqrt{d})V$ (7.2a)	Used verbatim in Layer 1
Sinusoidal positional encoding (7.2a)	Same Vaswani formula, scaled by embedding norm
Residual connection (7.2b)	Added to Layer 1 output (paper fn. 13)
Cosine similarity (Topic 7.1)	Importance score: $\cos(e_i, X^*)$
Encoder-style training	Supervised on returns , not next-token

- **Methodological pivot.** A standard LLM trains attention with an unsupervised objective (next-token cross-entropy). This paper trains the *same* attention machinery against a **financial target** (binary positive return). The attention weights then read directly as economic importance.
- **Importance score for paragraph i in document t :**

$$\text{Importance}_{ikt} = \frac{X_{it}^* \cdot e_{ikt}}{\|X_{it}^*\| \|e_{ikt}\|}$$

A paragraph is important when its semantic content aligns with the document's attention-weighted representation.

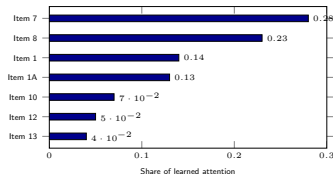
Empirical Findings

Predictive performance (avg AUC, 20 yearly test sets, baseline 0.50):

Model	AUC	Δ vs base
Attention (paper)	0.5599	+6.0 pp
MLP (no attn)	0.5328	+3.3 pp
Logit	0.5197	+2.0 pp

Sharpe (1-month-ahead returns): **1.56** attention, 1.28 MLP, 1.08 logit.

Attention share by 10-K Item (stylized; learned weights):



Top: Item 7 MD&A, Item 8 Financials. Bottom: Item 13

Topics inside MD&A (Item 7):

- *Most attended*: segment information, financial performance, liquidity, recent M&A.
- *Least attended*: forward-looking statements, accounting pronouncements, **corporate governance**, **sustainability / CSR**.

Surprise. ESG and corporate governance — dominant in public discourse — rank *lowest* by market reaction.

Two causal results.

- SEC *Modernization of Reg S-K* (treatment) raised Item 7 vs Item 8 (control) importance — regulation can *enhance* relevance.
- Firms strategically place bad-news / low-profitability content **later** in MD&A (Bloomfield 2008; Cohen et al. 2020).

Take-Aways and Bridges

1. **Same architecture, different “token.”** Wherever an ordered sequence has context-dependent importance, the transformer is a tool. Macro/finance natural fits:
 - Paragraphs of FOMC minutes / Beige Book (monetary-policy text)
 - Sentences of central-bank speeches (hawk–dove decomposition)
 - Stocks in an investor’s portfolio — Gabaix, Koijen, Richmond, Yogo (2023), *Asset Embeddings*
 - Country–year observations in a macro panel
2. **Supervised attention is a research instrument.** Once attention is trained against an economic target (returns, earnings, employment, output), the weights become inspectable measures of *economic* importance — decomposable across items, topics, time, and firm characteristics.
3. **Bridge to Topic 7.3a.** This paper is *encoder*-style supervised. Topic 7.3a covers *decoder-only* generative training. The same paper actually combines both: GPT-4o (decoder LLM) discovers the topic taxonomy that the encoder then scores — a hybrid pipeline the next topic equips you to build.